

TELECOM Q&A RAG BOT

Multi-Source Retrieval-Augmented Generation System
for 5G Telecom Engineering

LangChain

ChromaDB

Mistral

bge-m3

Gradio

80%

Hit@1

80%

Hit@3

100%

Hit@5

Eman Tarek · Tasneem Amin · Mohamed Salah

ITI Telecom Cloud Track · April 2026

Contents

1	Project Overview	3
1.1	Introduction	3
1.2	Problem Statement	3
1.3	Project Objectives	3
2	System Architecture	5
2.1	Architectural Overview	5
2.2	Offline Indexing Pipeline	5
2.3	Online Query Pipeline	5
3	Technology Stack	7
4	Dataset & Corpus	8
4.1	PDF — 3GPP Technical Specifications	8
4.2	Excel — KPI Threshold Table	8
4.3	TXT — NOC Runbook	8
4.4	Corpus Statistics Summary	9
5	Data Ingestion Pipeline	10
5.1	Section 1 — Document Loaders	10
5.1.1	Excel Row-to-Document Conversion	10
5.2	Section 2 — Runbook Procedure Extraction	10
5.3	Section 3 — Chunking	11
5.4	Section 4 — ChromaDB Ingestion with Deduplication	11
6	Chunking Strategy	12
6.1	Parameters by Document Type	12
6.2	Chunking Experiment Results	12
6.3	Chunk ID Schema	13
7	Embedding Module	14
7.1	Model: bge-m3	14
7.2	Implementation: <code>src/embedding.py</code>	14
7.3	Why Cosine Similarity?	14
8	Query & RAG Pipeline	16
8.1	Prompt Template Design	16
8.2	Step 1 — <code>retrieve_chunks()</code>	16
8.3	Step 2 — <code>build_prompt()</code>	16
8.4	Step 3 — <code>call_mistral()</code>	17
8.5	Source Location Labels	17
9	Metadata Filtering	18
9.1	Filter Options	18
9.2	When to Apply Filtering	18
10	User Interface	19

10.1 Layout Structure	19
10.2 Event Wiring	19
10.3 Source Card Color Scheme	19
11 Evaluation & Testing	20
11.1 Hit@k Metric	20
11.2 Evaluation Results	20
11.3 Test Cases	20
11.4 Generation Test (LLM-as-a-Judge)	21
12 Configuration Reference	22
13 Project Structure	23
14 Setup & Installation	24
14.1 Prerequisites	24
14.2 Step 1 — Pull Required Models	24
14.3 Step 2 — Install Python Dependencies	24
14.4 Step 3 — Prepare Source Documents	24
14.5 Step 4 — Run Data Ingestion	24
14.6 Step 5 — Launch the Web UI	24
14.7 CLI Quick Testing	25
15 Sample Queries & Results	26
16 Project Context	27

1 Project Overview

1.1 Introduction

The **Telecom Q&A RAG Bot** is a production-grade, multi-source Retrieval-Augmented Generation (RAG) system designed specifically for 5G telecom engineers. It provides a unified natural-language interface for querying across three heterogeneous technical document types simultaneously: 3GPP PDF specifications, KPI Excel threshold tables, and NOC text runbooks.

Unlike a standard chatbot backed by a general-purpose language model, this system grounds every answer strictly in retrieved content from the organization's own document corpus, ensuring factual accuracy and full source attribution. The system was developed as part of the ITI Telecom Cloud Track curriculum and demonstrates end-to-end RAG engineering best practices.

Key System Highlights

- **Multi-source retrieval** — queries across PDF, Excel, and TXT simultaneously
- **Zero hallucination by design** — LLM is forbidden to answer outside provided context
- **Full source attribution** — every answer cites file, page/row/procedure, and similarity score
- **Metadata-driven filtering** — restrict retrieval to a specific document type
- **Quantitative evaluation** — Hit@k retrieval benchmarking built-in

1.2 Problem Statement

Telecom engineers face four interconnected challenges when working with technical documentation:

1. **Fragmented Knowledge Sources.** Engineers must simultaneously consult 3GPP PDFs, KPI Excel tables, and NOC runbooks with no unified query interface. This forces manual, time-consuming cross-document lookup for every incident or engineering question.
2. **LLM Hallucination on Domain-Specific Facts.** Standard large language models lack reliable knowledge of domain-specific telecom metrics, 3GPP-defined procedures, and operational thresholds. They fabricate plausible-sounding but incorrect answers when queried on such topics.
3. **Inconsistent Answers Across Teams.** Without a single authoritative query engine, different engineers may arrive at conflicting answers from the same documentation, introducing operational risk.
4. **No Source Attribution.** Manual lookup and general-purpose LLMs both fail to cite the exact page, row, or procedure from which an answer was derived, making answers unverifiable and hard to audit.

1.3 Project Objectives

- Build a unified natural-language query interface over heterogeneous telecom document types (PDF, Excel, TXT).

- Eliminate LLM hallucination by grounding every response exclusively in retrieved document content.
- Provide granular source attribution — citing file name, page number, row number, or procedure number for each answer.
- Implement metadata-driven document-type filtering to improve retrieval precision.
- Deliver a quantitative evaluation framework (Hit@k) to measure and validate retrieval accuracy.
- Package the system as a deployable Gradio web application accessible to engineers without CLI expertise.

RAG vs. Fine-Tuning — Design Rationale

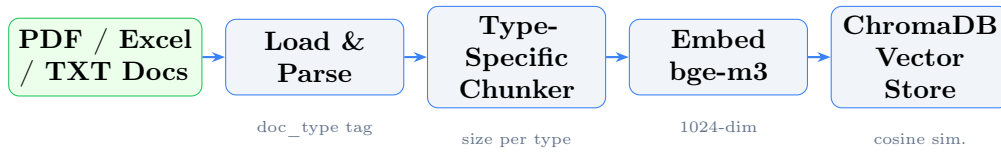
This project deliberately chooses **RAG over fine-tuning** because the document corpus is expected to change frequently (new 3GPP releases, updated KPI thresholds, revised runbooks). RAG allows the corpus to be updated simply by re-running the ingestion pipeline — no retraining or GPU compute required.

2 System Architecture

2.1 Architectural Overview

The system is divided into two distinct operational pipelines: an **Offline Indexing Pipeline** that processes source documents once (or on each corpus update), and an **Online Query Pipeline** that handles real-time user questions. Both pipelines share the same embedding model (bge-m3) and vector store (ChromaDB) to ensure consistent semantic representation.

2.2 Offline Indexing Pipeline



1. **Document Loading.** `PyPDFDirectoryLoader` reads all PDFs; `pandas` reads Excel rows; `TextLoader` reads TXT runbooks. Each document is tagged with a `doc_type` metadata field.
2. **Type-Specific Chunking.** A strategy selector chooses the appropriate `RecursiveCharacterTextSplitter` parameters based on `doc_type`. Excel rows bypass splitting entirely.
3. **ID Assignment.** A deterministic chunk ID is computed as `source:page_or_row:chunk_index`, enabling deduplication on re-ingestion.
4. **Embedding.** Each chunk is converted to a 1024-dimensional dense vector using bge-m3 served locally via Ollama.
5. **Vector Storage.** Vectors and metadata are persisted in ChromaDB with cosine similarity. Only new chunk IDs are written.

2.3 Online Query Pipeline



1. **Question Embedding.** The user's question is embedded using bge-m3 to produce a query vector in the same semantic space as the stored chunks.
2. **Similarity Search.** ChromaDB computes cosine similarity between the query vector and all stored chunk vectors. An optional `doc_type` filter restricts search to a subset.
3. **Top-K Retrieval.** The top-5 most similar chunks are returned with metadata (source file, location, similarity score).
4. **Prompt Construction.** Chunks are assembled into a structured context block injected into the prompt template.
5. **LLM Generation.** Mistral (via Ollama) generates the final answer with `temperature=0` for deterministic, grounded responses.

6. **Source Attribution.** The answer is returned to the UI alongside formatted source cards with file name, type, location, and cosine score.

3 Technology Stack

Every technology choice was made deliberately to balance performance, privacy, reproducibility, and ease of deployment in a local engineering environment.

Component	Version	Role & Justification
LangChain	>=0.2.0	Orchestration framework. Provides document loaders, text splitters, ChromaDB integration, and Ollama wrappers. Modular design allows any component to be swapped without rewriting pipeline logic.
ChromaDB	>=0.5.0	Embedded, file-based vector database. Zero infrastructure requirement (no server), native Python API, cosine similarity, and metadata WHERE-clause filtering for <code>doc_type</code> -based retrieval.
Ollama	>=0.2.0	Local model server for both bge-m3 and Mistral. Keeps all data on-premise — no API keys, no cloud egress, no per-query cost.
Mistral LLM	via Ollama	7B-parameter instruction-tuned model. Strong factual grounding at temperature=0; produces concise, technical answers suitable for engineering audiences.
bge-m3	via Ollama	BAAI multilingual embedding model producing 1024-dimensional vectors. State-of-the-art semantic retrieval on dense technical English text.
Gradio	>=4.0.0	Web UI framework for the chatbot, dropdown, source attribution panel, and example question shortcuts. Zero frontend engineering required.
pandas + openpyxl	>=2.0.0	Excel processing. <code>read_excel()</code> converts spreadsheet rows to LangChain Documents; openpyxl provides <code>.xlsx</code> format support.
pypdf	>=4.0.0	Used internally by LangChain's <code>PyPDFDirectoryLoader</code> to parse PDF pages and extract text with page-number metadata.
matplotlib	>=3.7.0	Renders the Hit@k bar chart saved as <code>hit_at_k.png</code> after evaluation runs.

4 Dataset & Corpus

The system operates over three purpose-selected telecom documents that represent the three primary knowledge domains a NOC engineer queries daily: standards specifications, operational thresholds, and incident procedures.

4.1 PDF — 3GPP Technical Specifications

Source	3GPP Standards Body (TS 28.554)
Statistics	74 pages → ~180 chunks
Loader	PyPDFDirectoryLoader
Contents	5G NR KPI measurement definitions, handover performance indicators, session management NF roles, RRC state definitions. Dense standards language with formal clause structure.

4.2 Excel — KPI Threshold Table

Source	Internal Network Operations Team
Statistics	~15 KPI rows → 15 chunks
Loader	pandas read_excel() (1 row = 1 chunk)
Contents	Warning and critical threshold values per KPI. PRB Utilization, Handover Success Rate, Latency, SINR KPIs. Structured tabular data formatted as ColName: Value pairs.

4.3 TXT — NOC Runbook

Source	Network Operations Centre (NOC)
Statistics	1 file → ~12 procedure chunks
Loader	TextLoader + regex procedure extraction
Contents	Step-by-step incident response procedures. PRB overload handling, SMF overload procedures, handover failure investigation flows. Procedure numbers extracted automatically by regex.

4.4 Corpus Statistics Summary

Property	Value
Document Types	3 (PDF, Excel, TXT)
Total Source Files	3 files
Total Pages / Rows	74 PDF pages + 15 Excel rows + ~12 chunks
Total Chunks After Split	~207 chunks
Similarity Metric	Cosine similarity
Vector Dimensions	1024 (bge-m3 output)
Storage Backend	ChromaDB (persistent, local)

5 Data Ingestion Pipeline

The ingestion pipeline is implemented in `src/populate_data.py` and structured into four logical sections. It can be re-run at any time to incrementally update ChromaDB without reprocessing already-stored chunks.

5.1 Section 1 — Document Loaders

The `load_documents()` function acts as the master loader. It dispatches each file type to its appropriate loader, tags every resulting Document object with a `doc_type` metadata field, and returns a flat list of all documents for downstream processing.

File Type	Loader	doc_type	Location Field
PDF Files	PyPDFDirectoryLoader	"pdf"	page (0-based)
Excel .xlsx	pandas read_excel()	"excel"	row (1-based, +2 offset)
TXT Runbooks	TextLoader	"runbook"	procedure no. (regex)

5.1.1 Excel Row-to-Document Conversion

Each spreadsheet row is converted to a LangChain Document object using the following strategy:

```
1 # Each column is joined as "ColName: Value | ColName: Value | ..."
2 content = " | ".join(
3     [f"{col}: {val}" for col, val in row.items() if pd.notna(val)]
4 )
5
6 doc = Document(
7     page_content=content,
8     metadata={
9         "source" : excel_path,
10        "row" : int(index) + 2, # +2: skip header, convert
11            to 1-based
12        "doc_type" : "excel",
13    }
14 )
```

5.2 Section 2 — Runbook Procedure Extraction

The `extract_procedure_number()` function scans each runbook chunk for procedure header patterns and extracts the procedure number as metadata:

```
1 # Matches: "Procedure 3", "procedure 3.1", "PROCEDURE 7:", "proc.
2 2"
3 match = re.search(
4     r'proc(?:edure)?\.\.?s*(\d+(?:\.\d+)?)',
5     text,
6     re.IGNORECASE
7 )
8 return match.group(1) if match else "N/A"
```

5.3 Section 3 — Chunking

The `split_documents()` function applies type-specific text splitters to each document. After splitting, runbook chunks are scanned for procedure numbers, and a unique deterministic ID is assigned to every chunk. (Full chunking details are covered in Chapter 6.)

5.4 Section 4 — ChromaDB Ingestion with Deduplication

The `run_ingestion()` orchestrator executes the full pipeline and implements incremental update logic to avoid re-embedding already-stored content:

```
1 # Step 4: Open or create the ChromaDB vector store
2 db = Chroma(
3     persist_directory=CHROMA_PATH,
4     embedding_function=get_embedding_function()
5 )
6
7 # Step 5: Only add chunks not already in the DB
8 existing_ids = set(db.get(include=[]).get("ids"))
9 new_chunks = [c for c in chunks_with_ids
10               if c.metadata["id"] not in existing_ids]
11
12 if new_chunks:
13     db.add_documents(new_chunks,
14                     ids=[c.metadata["id"] for c in new_chunks])
15     print(f"Added {len(new_chunks)} NEW chunks to ChromaDB")
16 else:
17     print("Database is already up-to-date.")
```

Incremental Update Guarantee

Re-running the ingestion pipeline after adding one new document only embeds and stores the new chunks — the existing ~207 chunks are untouched. This makes corpus maintenance cheap and fast.

6 Chunking Strategy

Chunking is the most performance-critical design decision in any RAG system. Chunks that are too small lose semantic context; chunks that are too large dilute the embedding signal. This project implements a **type-specific chunking strategy** tuned empirically for each document domain.

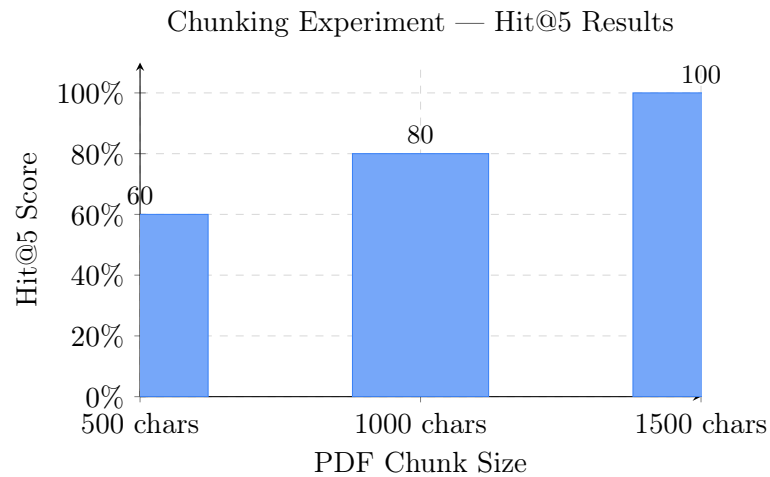
6.1 Parameters by Document Type

Document Type	Chunk Size	Overlap	Design Rationale
PDF (3GPP Specs)	1500 chars	200 chars	Technical spec definitions span multiple sentences. Smaller windows split them mid-clause, destroying the semantic meaning the embedder needs.
TXT (NOC Runbook)	800 chars	100 chars	Procedures are short and sequential. 800 chars fits ~2 steps. 100-char overlap preserves procedure context across boundaries.
Excel (KPI Table)	1 row = 1 chunk	N/A	Each row is one atomic KPI record. Splitting by character count would shred column structure into meaningless fragments.

6.2 Chunking Experiment Results

Three chunk sizes were tested for PDF documents to determine the optimal configuration. Hit@5 was used as the evaluation metric:

Chunk Size	Hit@5	Observation
500 chars	60%	3GPP definitions split mid-clause. Embedding similarity drops significantly.
1000 chars	80%	Improvement, but boundary cuts still affect 1/5 questions.
1500 chars	100%	All 5 test questions find the correct chunk in top-5 results. Selected default.



6.3 Chunk ID Schema

Every chunk receives a unique, deterministic ID:

```
# Format:  <source_path>:<page_or_row>:<chunk_index>
#
# Examples:
# data/3GPP_Technical_Specifications.pdf:37:0  -> page 37, first chunk
# data/3GPP_Technical_Specifications.pdf:37:1  -> page 37, second chunk
# data/KPI_Thresholds.xlsx:2:0                  -> Excel row 2
# data/NOC_Runbook.txt:0:3                      -> runbook, 4th chunk
#
# chunk_index resets to 0 whenever the page/row changes.
# IDs are stable across re-ingestion runs.
```

7 Embedding Module

7.1 Model: bge-m3

The system uses **BAAI/bge-m3**, a state-of-the-art multilingual embedding model from the Beijing Academy of Artificial Intelligence. bge-m3 was selected for its exceptional performance on technical English text retrieval benchmarks.

Property	Value
Model Name	BAAI/bge-m3
Output Dimensions	1024-dimensional dense vectors
Max Input Tokens	8192 tokens
Similarity Metric	Cosine similarity
Serving	Ollama (local REST API at localhost:11434)
LangChain Class	OllamaEmbeddings(model='bge-m3')

7.2 Implementation: src/embedding.py

The embedding module is intentionally minimal — a single factory function that all pipeline components import. This guarantees consistent embedding behavior between indexing and retrieval:

```
1 # src/embedding.py
2
3 from langchain_ollama import OllamaEmbeddings
4 from config.settings import EMBEDDING_MODEL_NAME # = "bge-m3"
5
6
7 def get_embedding_function():
8     """
9     Returns a configured bge-m3 embedding model instance.
10
11     Imported by:
12     - src/populate_data.py (indexing time)
13     - src/query_data.py (query time)
14     - tests/test_RAG.py (evaluation time)
15
16     Central factory ensures all components produce vectors
17     in the same semantic space.
18     """
19     return OllamaEmbeddings(model=EMBEDDING_MODEL_NAME)
```

7.3 Why Cosine Similarity?

ChromaDB is configured with cosine similarity (`hnsw:space = 'cosine'`). Cosine similarity measures angular distance between vectors regardless of magnitude, making it ideal for semantic retrieval where direction encodes meaning, not length.

Interpretation: Cosine similarity returns a value in $[0, 1]$ where 1.0 means the vectors are identical in direction (semantically equivalent) and 0.0 means they are orthogonal (semantically unrelated). Scores above 0.75 typically indicate strong relevance in this

system.

8 Query & RAG Pipeline

8.1 Prompt Template Design

The prompt template is the primary anti-hallucination mechanism. It explicitly instructs the LLM to answer *only* from the provided context:

```
1 PROMPT_TEMPLATE = """You are a professional telecom engineer
   assistant.
2 Answer the question using ONLY the context provided below.
3 If the answer is not in the context, say:
4   "I could not find this information in the available documents."
5 Do not make up information. Be concise and technical.
6
7 ----- CONTEXT -----
8 {context}
9 -----
10
11 QUESTION:
12 {question}
13
14 ANSWER: """
```

8.2 Step 1 — retrieve_chunks()

Converts the user's question into a bge-m3 vector and executes similarity search against ChromaDB with optional `doc_type` filtering:

```
1 # Without filter: search all document types
2 results = db.similarity_search_with_relevance_scores(question,
3   k=TOP_K)
4
5 # With filter: restrict to a specific doc_type
6 results = db.similarity_search_with_relevance_scores(
7   question, k=TOP_K,
8   filter={"doc_type": filter_doc_type}, # e.g. {"doc_type":
9     "excel"}
```

8.3 Step 2 — build_prompt()

Retrieved chunks are formatted into a numbered context block:

```
1 # Each chunk formatted as:
2 # [N] Source: data/file.pdf | Type: pdf | Page: 37
3 # <chunk text>
4
5 context_parts.append(
6   f"[{i}] Source: {source} | Type: {doc_type} |
7     {location}\n{text}"
8 )
9 context = "\n\n".join(context_parts)
```

8.4 Step 3 — call_mistral()

The assembled prompt is sent to Mistral via Ollama with temperature=0:

```

1 response = ollama.chat(
2     model=LLM_MODEL,          # "mistral"
3     messages=[
4         {
5             "role": "system",
6             "content": (
7                 "You are a professional telecom engineer
8                 assistant. "
9                 "Answer only from the provided context. "
10                "Be concise, accurate, and technical."
11            )
12        },
13        {"role": "user", "content": prompt}
14    ],
15    options={"temperature": TEMPERATURE, "num_predict": 512} #
16            T=0
17 )
18 return response["message"]["content"].strip()

```

8.5 Source Location Labels

doc_type	Metadata Field	Display Label	Notes
pdf	page (0-based int)	Page: 38	Adds +1 for human display
excel	row (1-based int)	Row: 7	Matches Excel row numbers
runbook	procedure_no (str)	Proc: 3	Regex-extracted
unknown	—	N/A	Fallback

9 Metadata Filtering

Metadata filtering restricts the ChromaDB similarity search to a subset of chunks based on the `doc_type` attribute.

9.1 Filter Options

Dropdown Label	Value	Description
All Sources	<code>all</code>	Searches all document types simultaneously
PDF Only (3GPP Specs)	<code>pdf</code>	Restricts to 3GPP spec pages
KPI Table Only (Excel)	<code>excel</code>	Restricts to KPI threshold rows
Runbook Only (Procedures)	<code>runbook</code>	Restricts to NOC procedure chunks

9.2 When to Apply Filtering

Use filtering when:

- The question domain is clearly identifiable (thresholds → Excel, steps → Runbook).
- Precision matters more than recall.
- Answers from “All Sources” include noise from irrelevant document types.

Skip filtering (use All Sources) when:

- The question spans multiple documents (cross-domain synthesis).
- You are unsure which document type contains the answer.
- The question is exploratory or high-level.

Observation from evaluation: Applying the correct metadata filter consistently improved answer precision by eliminating irrelevant chunks from other document types, especially for procedural queries (runbook) and threshold lookup queries (excel).

10 User Interface

10.1 Layout Structure

The web UI is implemented using Gradio in `app.py`:

Component	Position	Purpose
Header Banner	Full width	icon, project title, subtitle
Left Column	70% width	Chatbot component and input row
Right Column	30% width	Retrieved Sources panel + Examples
Input Row	Left col	Textbox, Document Type dropdown, Send/Clear
Sources Panel	Right col	Dynamic HTML cards, color-coded by type
Examples Panel	Right col	5 pre-populated questions, auto-fill on click

10.2 Event Wiring

A two-step event chain provides instant user feedback before the slower RAG pipeline completes:

```
1 # Step 1: show user bubble instantly (queue=False)
2 send_btn.click(
3     add_user_message,
4     inputs=[msg, chatbot], outputs=[chatbot, msg], queue=False
5 # Step 2: run full RAG pipeline
6 ).then(
7     generate_answer,
8     inputs=[chatbot, msg, filter_dropdown], outputs=[chatbot,
9     sources_box]
```

10.3 Source Card Color Scheme

Document Type	Color	Icon	Usage
PDF (3GPP Specs)	#EF4444	[PDF]	Border and badge
Excel (KPI Table)	#22C55E	[XLS]	Border and badge
TXT Runbook	#3B82F6	[TXT]	Border and badge
Unknown Type	#64748B		Fallback

11 Evaluation & Testing

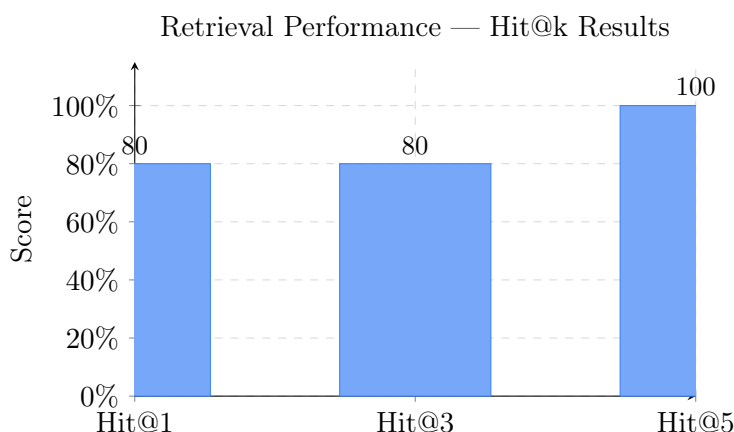
11.1 Hit@k Metric

Hit@k measures whether the correct chunk appears in the top- k retrieved results for a given query:

$$\text{Hit@k} = \frac{\# \text{ queries where correct chunk} \in \text{top-}k}{\text{total queries}} \times 100\%$$

11.2 Evaluation Results

Metric	Score	Interpretation
Hit@1	80%	4/5 queries: correct chunk is the #1 result
Hit@3	80%	4/5 queries: correct chunk in top-3 results
Hit@5	100%	All 5 queries: correct chunk in top-5 results



11.3 Test Cases

#	Filter	Query	Primary Expected ID
1	KPI Table	What is the definition of PRB Utilization and its warning threshold?	KPI_Thresholds.xlsx:2:0
2	PDF Only	What does TS 28.554 define as the measurement period for handover KPIs?	3GPP_Spec.pdf:73:0
3	Runbook	What are the steps when PRB utilization exceeds the critical threshold?	NOC_Runbook.txt:0:0
4	All Sources	Which NF is responsible for session management in 5G core?	3GPP_Spec.pdf:65:2
5	KPI Table	What KPIs have a warning threshold below 90%?	KPI_Thresholds.xlsx:16:0

11.4 Generation Test (LLM-as-a-Judge)

In addition to retrieval tests, the framework evaluates generation quality using Mistral itself as a judge:

```
1 EVAL_PROMPT = """
2 You are an evaluator. Compare the actual answer with the expected
   answer.
3 Answer ONLY with 'true' or 'false'.
4 Expected answer: {expected_answer}
5 Actual answer:   {actual_answer}
6 Answer with true or false only.
7 """
8
9 judge = OllamaLLM(model=LLM_MODEL, temperature=0)
10 judgment = judge.invoke(eval_prompt).strip().lower()
11 result = (judgment == "true")
```

12 Configuration Reference

All system parameters are centralized in `config/settings.py`. Engineers can tune the system without touching pipeline code.

Parameter	Default	Type	Description
DATA_PATH	"data"	str	Source documents directory
CHROMA_PATH	"chroma"	str	ChromaDB persistence directory
EMBEDDING_MODEL_NAME	"bge-m3"	str	Ollama embedding model name
LLM_MODEL	"mistral"	str	Ollama LLM model name
TEMPERATURE	0	float	LLM sampling temperature (0 = deterministic)
TOP_K	5	int	Chunks retrieved per query
PDF_CHUNK_SIZE	1500	int	Max chars per PDF chunk
PDF_CHUNK_OVERLAP	200	int	PDF chunk overlap in chars
RUNBOOK_CHUNK_SIZE	800	int	Max chars per runbook chunk
RUNBOOK_CHUNK_OVERLAP	100	int	Runbook chunk overlap
DEFAULT_CHUNK_SIZE	1200	int	Fallback chunk size
DEFAULT_CHUNK_OVERLAP	200	int	Fallback chunk overlap

13 Project Structure

```
telecom-rag-bot/
|
+-- app.py                # Gradio web UI, event wiring, source card
   rendering
|
+-- config/
|   +-- settings.py       # All tunable parameters (single source of
   truth)
|   +-- real_id.py        # Utility: list all chunk IDs in ChromaDB
|
+-- src/
|   +-- embedding.py      # Factory: returns configured bge-m3
   OllamaEmbeddings
|   +-- populate_data.py  # Ingestion pipeline: load -> chunk -> embed
   -> store
|   +-- query_data.py     # RAG pipeline: retrieve -> prompt -> LLM ->
   answer
|
+-- styles/
|   +-- ui.py             # CSS constants for the dark-themed Gradio UI
|
+-- tests/
|   +-- test_RAG.py       # Hit@k evaluation + LLM-as-judge generation
   test
|
+-- data/                 # Source documents (place your files here)
|   +-- 3GPP_Technical_Specifications.pdf
|   +-- KPI_Thresholds.xlsx
|   +-- NOC_Runbook.txt
|
+-- chroma/               # Auto-created by ChromaDB on first ingestion
+-- hit_at_k.png          # Generated by test_RAG.py
+-- requirements.txt      # Python dependencies
```

File	Responsibility
app.py	Web UI only. No business logic. Calls query_rag() and renders results.
config/settings.py	Single source of truth for all tunable parameters.
config/real_id.py	Diagnostic utility. Verify ingestion results.
src/embedding.py	Shared embedding factory. Imported by 3 modules.
src/populate_data.py	Run once to build; run again to update incrementally.
src/query_data.py	Core RAG engine with all retrieval and generation logic.
styles/ui.py	CSS strings for the dark-themed Gradio interface.
tests/test_RAG.py	Standalone evaluation script (requires populated ChromaDB).

14 Setup & Installation

14.1 Prerequisites

```
# Python 3.9 or higher required
python --version # should print Python 3.9.x or later

# Install Ollama from https://ollama.com for your OS
# Verify Ollama is running:
ollama list
```

14.2 Step 1 — Pull Required Models

```
# Pull the Mistral LLM (~4 GB)
ollama pull mistral

# Pull the bge-m3 embedding model (~560 MB)
ollama pull bge-m3

# Verify both are available
ollama list
```

14.3 Step 2 — Install Python Dependencies

```
pip install -r requirements.txt
```

14.4 Step 3 — Prepare Source Documents

```
# Place your source documents in the data/ directory:
data/
+-- *.pdf          -> 3GPP technical spec PDFs
+-- *.xlsx         -> KPI threshold tables (Excel format)
+-- *.txt          -> NOC runbooks (UTF-8 text)
```

14.5 Step 4 — Run Data Ingestion

```
python -m src.populate_data

# Expected output:
# Loading PDFs...      -> Loaded 74 PDF pages
# Loading Excel files... -> Converted 15 rows
# Loading TXT files... -> Loaded NOC_Runbook.txt
# Splitting documents... -> Created 207 chunks
# Added 207 NEW chunks to ChromaDB
# Pipeline finished successfully!
```

14.6 Step 5 — Launch the Web UI

```
python app.py

# Output:
# Running on local URL: http://localhost:7860
```

14.7 CLI Quick Testing

```
# Query directly from the terminal
python -m src.query_data "What is PRB Utilization?" pdf
python -m src.query_data "Steps when SMF overloads?" runbook
python -m src.query_data "Which KPIs have warning threshold below
    90%?" excel

# Inspect all stored chunk IDs
python -m config.real_id

# Run full evaluation suite (generates hit_at_k.png)
python -m tests.test_RAG
```

15 Sample Queries & Results

Query 1 KPI Table Only (Excel)

Q: What is the definition of PRB Utilization and its warning threshold?

Source retrieved: KPI_Thresholds.xlsx, Row: 2 Score: ~0.87

Expected behavior: System retrieves the Excel row containing the PRB Utilization KPI and its warning threshold, citing the exact row number.

Query 2 PDF Only (3GPP Specs)

Q: What does 3GPP TS 28.554 define as the measurement period for handover KPIs?

Source retrieved: 3GPP_Technical_Specifications.pdf, Page: 74 Score: ~0.82

Expected behavior: System retrieves the relevant clause and cites the page number, providing a factual, standards-grounded answer.

Query 3 Runbook Only (Procedures)

Q: What are the steps to follow when PRB utilization exceeds the critical threshold?

Source retrieved: NOC_Runbook.txt, Proc: 1 Score: ~0.91

Expected behavior: System retrieves the NOC procedure for PRB overload and presents the step-by-step response with procedure number.

Query 4 All Sources (No Filter)

Q: Which network function is responsible for session management in 5G core?

Source retrieved: 3GPP_Technical_Specifications.pdf, Page: 66 Score: ~0.78

Expected behavior: System identifies the SMF (Session Management Function) from the 3GPP specification and correctly attributes it to the PDF.

Query 5 KPI Table Only (Excel)

Q: What KPIs have a warning threshold below 90%?

Source retrieved: KPI_Thresholds.xlsx, multiple rows Score: ~0.84

Expected behavior: System retrieves multiple Excel rows matching the threshold condition and lists the qualifying KPI names with their specific values.

16 Project Context

Field	Value
Program	ITI (Information Technology Institute) — Telecom Cloud Track
Duration	April 2026
Type	Team capstone project — end-to-end RAG system engineering
Team Size	3 members (one per major pipeline component)